Gradient Descent

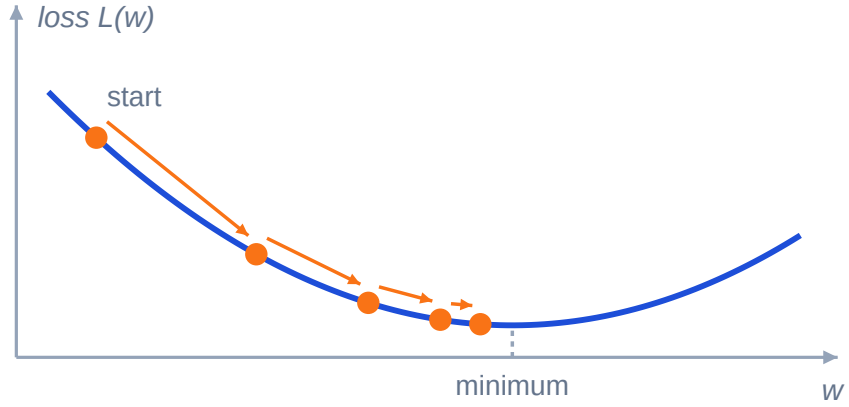
How a model learns by walking downhill

1. The idea

We want the weights w that make the loss L(w) as small as possible. The gradient ∇L(w) points uphill, so we take a small step the other way and repeat until the loss stops falling.

$$w \leftarrow w - \eta \cdot \nabla L(w)$$

η is the learning rate: how big each step is.



2. The loop

1. Start from a random guess for w.
2. Compute the gradient ∇L(w) on a batch of data.
3. Update the weights: w ← w – η · ∇L(w).
4. Repeat until the loss stops falling.

KEY TAKEAWAYS

- Step against the gradient, not along it.
- Too large η: the loss bounces or explodes.
- Too small η: training crawls.
- Scale your features first.

Choosing the Learning Rate

One number decides whether training works

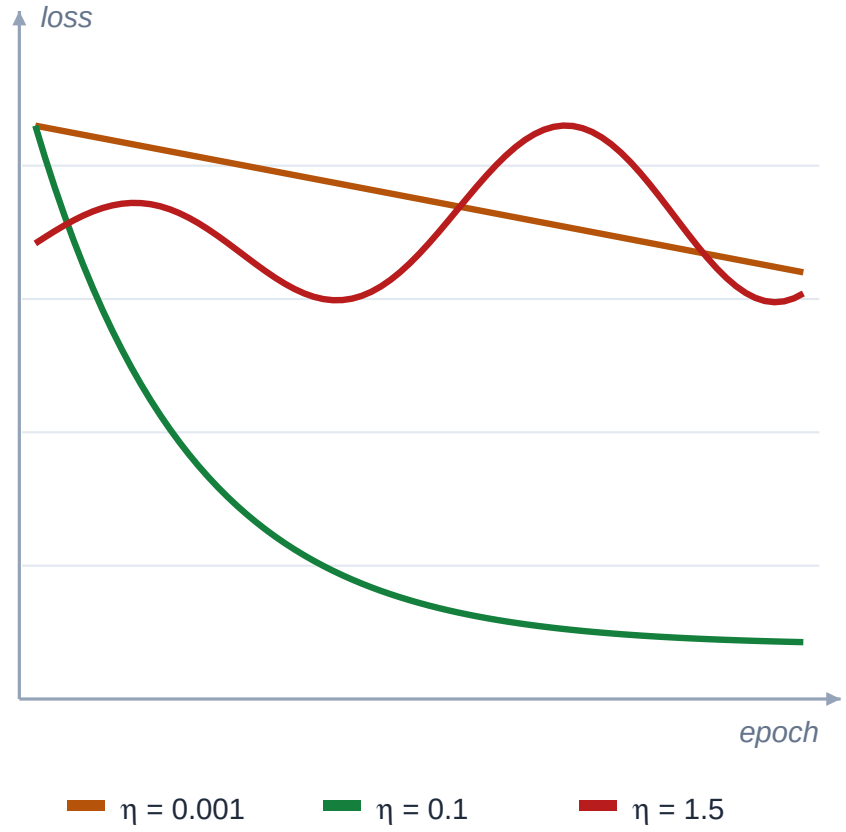
3. Too small, just right, too large

Each step moves the weights by η times the gradient. Small steps are safe but slow; large steps overshoot the minimum.

Learning rate η	What you see	Verdict
`0.001`	Loss falls very slowly	**Too small**
`0.1`	Loss falls fast and smoothly	**Good start**
`1.5`	Loss jumps around and grows	**Too large**

Tip

Start at 0.1. If the loss bounces, divide by 10. If it barely moves after a few epochs, multiply by 3.



Summary

Gradient descent in four lines

1. Compute the gradient of the loss with respect to the weights.
2. Move the weights a small step against the gradient.
3. Pick η with care: too large bounces, too small crawls.
4. Repeat over many batches until the loss levels off.

```
for epoch in range(100):
    grad = gradient(loss, w)
    w = w - lr * grad
```

Next lecture:
Momentum and Adam